Support Vector Machines: An Introduction

AI 500 - Foundations of AI

Fall 2025

Contents

1 The Core Idea **2**

2 Visual Intuition **2**

3 The Mathematics **2**

4 Worked Example **3**

5 How Training Works **6**

6 Support Vectors **7**

7 Beyond Linear Boundaries: The Kernel Trick **7**

8 Summary **7**

9 Additional Practice Problems **9**

9.1 Problem 1: Manual Weight Update Iteration 9

9.2 Problem 2: Computing the Margin Width 10

1 The Core Idea

Imagine you have data points from two classes (say, red circles and blue squares) scattered on a page. Your job is to draw a line separating them.

The question is: Which line is best?

Many lines could separate the two classes, but the **Support Vector Machine (SVM)** finds the line that is **safest**; or the one that stays as far away as possible from both classes.

Key Concepts

Term	Meaning
Hyperplane	The decision boundary separating classes (a line in 2D, a plane in 3D, etc.)
Margin	The "safety zone"; the distance from the hyperplane to the nearest points
Support Vectors	The critical points closest to the hyperplane that define the margin
Goal	Maximize the margin while correctly classifying all points

2 Visual Intuition

The **margin** is the width of the "safety zone" on either side of the hyperplane. SVM adjusts the boundary until this margin is as wide as possible. The data points that touch the margin boundaries are called **support vectors**: they "support" or define the optimal position of the hyperplane.

3 The Mathematics

For a linear boundary in 2D, we can write:

$$\mathbf{w} \cdot \mathbf{x} - b = 0$$

where:

- $\mathbf{w}$ = weight vector (perpendicular to the hyperplane)
- $\mathbf{x}$ = data point coordinates
- b = bias (shifts the hyperplane)

The Margin Constraint

For each data point (x_i, y_i) where $y_i \in \{-1, +1\}$:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1$$

This ensures:

- Points from class $+1$ satisfy: $\mathbf{w} \cdot \mathbf{x} - b \geq 1$

- Points from class -1 satisfy: $\mathbf{w} \cdot \mathbf{x} - b \leq -1$

The margin width is: $\frac{2}{\|\mathbf{w}\|}$
Therefore, maximizing the margin means **minimizing** $\|\mathbf{w}\|^2$.

Hinge Loss

What if a point violates the margin or is misclassified? We use **hinge loss**:

$$\ell_i = \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i - b))$$

- If a point is correctly classified and outside the margin: loss $= 0$
- If a point is too close or misclassified: loss > 0

The Complete Objective Function

$$J = \lambda\|\mathbf{w}\|^2 + \frac{1}{n}\sum \ell_i$$

This balances:

1. $\lambda\|\mathbf{w}\|^2 \rightarrow$ Maximize the margin (keep $\mathbf{w}$ small)
2. $\sum \ell_i \rightarrow$ Minimize classification errors

4 Worked Example

Let's solve a simple 2D classification problem by hand.

Given Data

Point	x_1	x_2	**Class** (y)
A	2	3	$+1$
B	3	3	$+1$
C	1	1	-1
D	2	1	-1

Step 1: Visualize the Data

We can see the classes are separable. We need a line between them.

Step 2: Try a Candidate Hyperplane

Let's try the line: $-x_1 + x_2 - 1 = 0$
This can be rewritten as: $x_2 = x_1 + 1$
So our weights are: $\mathbf{w} = (-1, 1)$ and $b = 1$
The decision function is: $f(\mathbf{x}) = -x_1 + x_2 - 1$

Step 3: Test Each Point

For each point, we calculate $f(\mathbf{x})$ and check if $y \cdot f(\mathbf{x}) \geq 1$.

Point	Calculation	$f(\mathbf{x})$	$y \times f(\mathbf{x})$	Margin Satisfied?	Hinge Loss
A(2,3)	$-2+3-1$	0	$(+1)(0)=0$	No (need ≥ 1)	$\max(0, 1-0)=1$
B(3,3)	$-3+3-1$	-1	$(+1)(-1)=-1$	No	$\max(0, 1-(-1))=2$
C(1,1)	$-1+1-1$	-1	$(-1)(-1)=1$	Yes	$\max(0, 1-1)=0$
D(2,1)	$-2+1-1$	-2	$(-1)(-2)=2$	Yes	$\max(0, 1-2)=0$

Step 4: Calculate Total Cost

Assuming $\lambda = 0.01$:

$$\|\mathbf{w}\|^2 = (-1)^2 + (1)^2 = 2$$

$$\text{Average hinge loss} = \frac{1+2+0+0}{4} = 0.75$$

$$J = 0.01(2) + 0.75 = 0.77$$

This hyperplane has high loss! Points A and B are too close or misclassified.

Step 5: Update the Weights

Using gradient descent, we'd adjust $\mathbf{w}$ and b to reduce the loss. After many iterations, the algorithm converges to optimal weights that maximize the margin while minimizing errors.

Derivation of the SVM Weight Update Rule

We begin with the regularized **hinge loss objective** for a linear SVM:

$$J(\mathbf{w}, b) = \lambda \|\mathbf{w}\|^2 + \frac{1}{n} \sum_{i=1}^{n} \max\left(0,\, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i - b)\right)$$

where:

- λ controls the strength of regularization (margin width),
- $\mathbf{w}$ is the weight vector,
- b is the bias,
- $y_i \in \{-1, +1\}$ are class labels.

Note on Notation: In practical implementations using stochastic gradient descent (SGD), we often see the cost written without the summation:

$$J = \lambda\|\mathbf{w}\|^2 + \frac{1}{n}\max\left(0, 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i - b)\right)$$

This represents the **per-sample cost** for a single training example during SGD, where we process one sample at a time. The summation form above represents the **total cost over the entire dataset**. Both formulations are equivalent, the difference is just whether we're looking at the full batch objective or the per-sample update during iteration.

Case 1: Correctly Classified Point (Outside the Margin)

A data point is correctly classified and lies outside the margin if:

$$y_i(\mathbf{w}\cdot\mathbf{x}_i - b) \geq 1$$

In this case, the hinge loss term is zero:

$$\max\left(0, 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i - b)\right) = 0$$

Therefore, only the regularization term contributes to the gradient:

$$\frac{\partial J}{\partial \mathbf{w}} = 2\lambda\mathbf{w}$$

The update rule using learning rate η becomes:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta(2\lambda\mathbf{w})$$

Intuition: If a point is correctly classified and far enough from the boundary, we don't need to adjust the line to fix errors, we only shrink the weights to keep the margin wide and prevent overfitting.

Case 2: Misclassified Point (Inside the Margin or on the Wrong Side)

A data point violates the margin if:

$$y_i(\mathbf{w}\cdot\mathbf{x}_i - b) < 1$$

Here, the hinge loss is active:

$$\ell_i = 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i - b)$$

The total cost for this point is:

$$J_i = \lambda\|\mathbf{w}\|^2 + \ell_i$$

Taking the gradient with respect to $\mathbf{w}$ and b:

$$\frac{\partial J_i}{\partial \mathbf{w}} = 2\lambda\mathbf{w} - y_i\mathbf{x}_i$$
$$\frac{\partial J_i}{\partial b} = y_i$$

The update rules become:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta(2\lambda\mathbf{w} - y_i\mathbf{x}_i)$$
$$b \leftarrow b - \eta y_i$$

Intuition: If a point is too close to or on the wrong side of the margin, the model shifts the hyperplane to reduce that point's loss.

- The $-y_i\mathbf{x}_i$ term pushes the boundary in the direction that moves the point to the correct side.
- The $2\lambda\mathbf{w}$ term still keeps the weights small for a wide margin.

Geometric Intuition

The two cases work together:

- **Case 1** gently contracts the weights (widening the margin).
- **Case 2** moves the separating line toward the misclassified point to reduce its error.

Over many iterations, this process converges to the hyperplane with the largest possible margin and minimal classification error.

5 How Training Works

1. **Initialize** $\mathbf{w}$ and b randomly (or to zeros)
2. **For each training point:**
   - Check if $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1$
   - If violated, update $\mathbf{w}$ and b to reduce hinge loss (Case 2)
   - If satisfied, only apply regularization (Case 1)
3. **Repeat** until convergence or max iterations reached
4. **Result:** The optimal separating hyperplane with maximum margin

Note on Objectives: During training, we can either:

- Use **batch gradient descent**: compute the total cost $J = \lambda\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n}\ell_i$ over all samples, then update once
- Use **stochastic gradient descent (SGD)**: process one sample at a time with per-sample cost $J_i = \lambda\|\mathbf{w}\|^2 + \ell_i$, updating after each sample

SGD is more common in practice as it's faster and works well with large datasets.

Making Predictions After Training

Once training is complete and we have optimal $\mathbf{w}$ and b, we classify new points using the **sign function**:

$$y_{\text{predicted}} = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b) = \begin{cases} +1 & \text{if } \mathbf{w} \cdot \mathbf{x} - b > 0 \\ -1 & \text{if } \mathbf{w} \cdot \mathbf{x} - b < 0 \end{cases}$$

Example: If $\mathbf{w} = (-0.798, 1.298)$, $b = 0.9$, and we want to classify point $(3, 4)$:

$$f(3,4) = -0.798(3) + 1.298(4) - 0.9 = 1.898$$
$$y_{\text{predicted}} = \text{sign}(1.898) = +1$$

During training, we use the raw value $f(\mathbf{x})$ to compute gradients. For final predictions, we apply sign to get the class label.

6 Support Vectors

After training, most points don't matter anymore! Only the **support vectors**; the points closest to the margin; determine the final hyperplane. In our example, these would be the points from each class that end up exactly on the margin boundaries.

7 Beyond Linear Boundaries: The Kernel Trick

What if the data isn't linearly separable (e.g., concentric circles)?

Solution: Transform the data into a higher-dimensional space where it becomes separable.

Common Kernels:

- **Linear:** $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- **Polynomial:** $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + r)^d$
- **RBF (Gaussian):** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$

8 Summary

Concept	Key Takeaway
SVM Goal	Find the hyperplane that maximizes the margin between classes
Support Vectors	The critical points that define the margin
Hinge Loss	Penalizes points that are too close or misclassified
Training	Balance between large margin (small $\|\mathbf{w}\|$) and low error
Kernel Trick	Transform data to higher dimensions for non-linear boundaries

Why SVM is powerful:

- Finds the "safest" boundary with maximum separation

- Only depends on support vectors (robust to outliers)
- Can handle non-linear boundaries with kernels
- Works well even with high-dimensional data

Based on your AI 500 lecture slides, please go through those as well for better clarity

9 Additional Practice Problems

9.1 Problem 1: Manual Weight Update Iteration

Given the following setup from our earlier example:

- Current weights: $\mathbf{w} = (-1, 1)$, $b = 1$
- Learning rate: $\eta = 0.1$
- Regularization: $\lambda = 0.01$
- Point A: $(2, 3)$ with label $y = +1$

We found that point A has $f(\mathbf{x}) = 0$, so $y \cdot f(\mathbf{x}) = 0 < 1$ (violates margin).
Task: Perform one weight update for this misclassified point.

Solution:

Since $y_A \cdot f(\mathbf{x}_A) = 0 < 1$, we use Case 2 (misclassified point):
Step 1: Calculate the gradient components

$$\begin{aligned} 2\lambda\mathbf{w} &= 2(0.01)(-1, 1) = (-0.02, 0.02) \\ y_A\mathbf{x}_A &= (+1)(2, 3) = (2, 3) \end{aligned}$$

Step 2: Update weights

$$\begin{aligned} \mathbf{w}_{\text{new}} &= \mathbf{w} - \eta(2\lambda\mathbf{w} - y_A\mathbf{x}_A) \\ &= (-1, 1) - 0.1((-0.02, 0.02) - (2, 3)) \\ &= (-1, 1) - 0.1(-2.02, -2.98) \\ &= (-1, 1) - (-0.202, -0.298) \\ &= (-1, 1) + (0.202, 0.298) \\ &= (-0.798, 1.298) \end{aligned}$$

Step 3: Update bias

$$\begin{aligned} b_{\text{new}} &= b - \eta y_A \\ &= 1 - 0.1(+1) \\ &= 0.9 \end{aligned}$$

Step 4: Verify improvement
New decision function: $f(\mathbf{x}) = -0.798x_1 + 1.298x_2 - 0.9$
For point A(2,3):

$$\begin{aligned} f(2, 3) &= -0.798(2) + 1.298(3) - 0.9 \\ &= -1.596 + 3.894 - 0.9 \\ &= 1.398 \end{aligned}$$

Check: $y_A \cdot f(\mathbf{x}_A) = (+1)(1.398) = 1.398 > 1$
The point is now correctly classified and outside the margin! The hinge loss for this point dropped from 1.0 to 0.

9.2 Problem 2: Computing the Margin Width

Consider a trained SVM with final weights $\mathbf{w} = (3, 4)$ and $b = 5$.

Task: Calculate the margin width and interpret what it means.

Solution:

The margin width formula is:

$$\text{Margin width} = \frac{2}{\|\mathbf{w}\|}$$

Step 1: Calculate $\|\mathbf{w}\|$

$$\begin{aligned}\|\mathbf{w}\| &= \sqrt{w_1^2 + w_2^2} \\ &= \sqrt{3^2 + 4^2} \\ &= \sqrt{9 + 16} \\ &= \sqrt{25} \\ &= 5\end{aligned}$$

Step 2: Calculate margin width

$$\text{Margin width} = \frac{2}{5} = 0.4$$

Interpretation:
The margin width of 0.4 means:

- The "safety zone" between the two classes has a total width of 0.4 units
- Each support vector is 0.2 units away from the decision boundary
- Points must be at least this far from the boundary to be considered correctly classified with confidence

What if we want a larger margin?
To increase the margin width, we need to decrease $\|\mathbf{w}\|$. For example:

- If $\mathbf{w} = (1.5, 2)$, then $\|\mathbf{w}\| = 2.5 \rightarrow$ margin $= \frac{2}{2.5} = 0.8$
- If $\mathbf{w} = (6, 8)$, then $\|\mathbf{w}\| = 10 \rightarrow$ margin $= \frac{2}{10} = 0.2$

This illustrates why the SVM objective minimizes $\|\mathbf{w}\|^2$ smaller weights mean a larger, safer margin!